

Hospilot Full-Stack Assessment

Part 1 - Implementation Plan & Interview Q&A

Candidate folder: widget/prerakyadav/

Document name: Hospilot_Part1_Implementation_Interview_QnA.pdf

Purpose: Explain how Part 1 was built, what technologies were used, the architecture and request flow, and prepare interview-style answers a reviewer or hiring manager might ask.

Scope: Part 1 only (Widget + Live Plan Viewer). Part 2 / RAG is out of scope.

1. Implementation Plan (How I Built It)

I followed the assessment contract in order: inspect the repo first, copy demo.html into a candidate folder, build a backend proxy, wire the frontend widget, harden iframe postMessage, add tests, then prepare deployment.

- Step 1: Inspect repo: widget/demo.html, README, .gitignore, assessment instructions.
- Step 2: Create widget/prerakyadav/ and copy demo.html (never edit the original demo in place).
- Step 3: Centralize CANDIDATE_NAME so every goal is prefixed [CANDIDATE-prerakyadav].
- Step 4: Build reusable Hospilot client: login, createSession, getSession.
- Step 5: Expose same-origin APIs: POST /api/session and GET /api/session/:id.
- Step 6: Validate goals server-side (trim, min/max length, reject empty/oversized).
- Step 7: Always create sessions with autonomous: false.
- Step 8: Frontend state machine: IDLE -> SUBMITTING -> PLANNING -> READY -> LOADING_IFRAME -> VIEWING_PLAN / ERROR.
- Step 9: Poll until pipeline.agents is non-empty (or timeout).
- Step 10: View Plan opens iframe to https://hospilot.carer.ai, waits for load, then postMessage widget_init.
- Step 11: Retry postMessage because Hospilot is an SPA and onload can fire before its listener is ready.
- Step 12: Add unit tests with mocked fetch + one controlled live integration check.
- Step 13: Prepare Vercel deployment (api/ serverless + static demo.html) and document env vars.

2. Things Used (Tech Stack & Tools)

Languages & runtime

- JavaScript (Node.js >= 18) for backend and local server
- Vanilla HTML / CSS / JS for the widget (no React/Vue)
- Python fpdf2 only to generate this interview PDF (not part of runtime)

Libraries

- dotenv - load credentials locally from .env
- Node built-in http + fetch - no Express required
- node:test + node:assert - zero-config unit tests

External systems

- Hospilot API: https://hospilot.carer.ai
- POST /api/auth/login
- POST /api/sessions (goal, constraints, autonomous:false)
- GET /api/sessions/{session_id}
- Hospilot web app in iframe + window.postMessage handoff

Deployment target

- Vercel: static frontend + /api serverless functions
- Platform environment variables (never commit secrets)

Key files

- demo.html - CarePlus HIS mock dashboard with widget wired in
- widget.js / widget.css - UI, state machine, iframe handoff
- server.js - local same-origin server
- api/session/index.js + api/session/[id].js - Vercel routes
- lib/hospilotClient.js - centralized upstream API client
- lib/handlers.js - request validation + response shaping
- lib/validate.js - goal rules + pipeline readiness detection
- tests/*.test.js - mocked API / validation tests
- scripts/integration-check.js - one real sandbox session check
- README.md - architecture, setup, security, tradeoffs

3. Architecture

Two separate paths exist on purpose:

```
Browser (widget)
  | same-origin HTTP
  v
Candidate Backend (/api/session)
  | Authorization: Bearer <token>
  v
Hospilot API (https://hospilot.carer.ai)
```

AND separately:

```
Browser
-> iframe src=https://hospilot.carer.ai
-> onload (+ short retries)
-> postMessage({ type: 'widget_init', token, sessionId })
```

Why split? Hospilot rejects browser-origin CORS for its API. A real hospital integration also keeps credentials and API access on the server. The browser only owns UI + the iframe handoff.

4. Exact Request Flow (What Reviewers Check)

- 1. User types a natural-language goal.
- 2. Frontend POST `/api/session { goal }` to OUR backend only.
- 3. Backend logs into Hospilot with env credentials.
- 4. Backend creates session with `[CANDIDATE-prerakyadav]` prefix and `autonomous:false`.
- 5. Response returns `sessionId + token` (token kept in memory only).
- 6. Frontend polls `GET /api/session/:id` every ~2.5s until `ready=true`.
- 7. Ready means `pipeline.agents` is non-empty.
- 8. UI enables View Plan.
- 9. View Plan shows iframe, waits for load, sends exact `postMessage` shape.
- 10. Iframe should show THAT session's plan - not an empty No Pipeline Yet screen.

5. Security Decisions

- Credentials only in env vars (.env locally, platform env in production).
- .env is gitignored; never put username/password in HTML/JS/README.
- Browser never calls Hospilot login/sessions APIs directly.
- Token returned once for iframe init; runtime memory only.
- Logs never include password, Authorization header, or full token.
- API errors return structured JSON codes - no stack traces to users.

6. Interview-Style Q&A (Full Prep)

Q: Walk me through what you built for Part 1.

A: I embedded a Hospilot workflow widget into a copied hospital HIS dashboard. The user enters a goal; my backend authenticates to Hospilot, creates a planning session with `autonomous=false` and a candidate prefix, polls until the pipeline is ready, then the UI opens the real Hospilot app in an iframe and initializes the exact session via `postMessage`.

Q: Why didn't you call the Hospilot API from the browser?

A: Two reasons: Hospilot CORS only allows known origins, so localhost browser calls fail by design; and real integrations keep credentials/API access on a trusted backend. The assessment explicitly requires a tiny backend proxy.

Q: Where do credentials live?

A: Only in environment variables: `HOSPILOT_USERNAME` and `HOSPILOT_PASSWORD`. Locally via `.env` (gitignored). In production via Vercel project settings. They are never committed and never sent to the frontend.

Q: Why does the frontend still receive a token?

A: Because the iframe handoff contract requires `postMessage({ type:'widget_init', token, sessionId })`. The token is required by Hospilot's UI to open the session. I return it only over HTTPS from our API and keep it in memory - not localStorage, cookies, or query params.

Q: What is the exact postMessage contract?

A: Exactly three fields: `type: 'widget_init'`, `token: '<jwt>'`, `sessionId: '<uuid>'`. No extra properties, no renamed keys. Sent only after iframe load (with retries because Hospilot is a SPA).

Q: Why autonomous must be false?

A: `autonomous:true` tells Hospilot to plan and immediately execute without human approval. This assessment is about planning/viewing the pipeline only, so we always send `autonomous:false`.

Q: How do you know the plan is ready?

A: I poll `GET /api/sessions/{id}`. Readiness is when pipeline has content - in practice pipeline is an object like `{ edges: [], agents: [...] }`. Empty scaffold `{agents:[]}` is NOT ready. Status alone can stay 'pending' even after agents exist, so I key off `pipeline.agents.length`.

Q: How do you avoid creating too many sessions?

A: Disable Generate Plan while SUBMITTING/PLANNING, ignore duplicate clicks in those states, and avoid auto-submit. Automated tests mock Hospilot. Only one controlled live integration script is used when needed.

Q: Describe your frontend state machine.

A: IDLE, SUBMITTING, PLANNING, READY, LOADING_IFRAME, VIEWING_PLAN, ERROR. View Plan is disabled until READY. Input is disabled while busy. ERROR shows a human message and Try again. This prevents contradictory UI and race conditions.

Q: What went wrong with the first iframe test and how did you fix it?

A: Auth worked (user appeared logged in) but the UI showed 'No Pipeline Yet'. Root cause: we briefly navigated the iframe to `about:blank`; onload fired too early and `postMessage` was marked sent before Hospilot's listener was ready. Fix: ignore `about:blank` loads and retry `widget_init` at 0/400/1000/2000/3500ms after real load.

Q: How is the candidate name handled?

A: CANDIDATE_NAME env/config defaults to prerakyadav. withCandidatePrefix() prepends [CANDIDATE-prerakyadav] unless already present. Centralized so it is easy to change.

Q: How did you validate user input?

A: Backend checks goal exists, is a string, trims whitespace, enforces min/max length (3..2000), rejects empty/whitespace and oversized payloads (body capped ~32KB). Natural language is not over-restricted.

Q: What HTTP status codes does your API use?

A: 201 on session create; 400 invalid goal/JSON; 401/403 upstream auth issues; 404 session not found; 413 payload too large; 429 rate limit; 502 upstream errors; 504 request timeout. Errors look like { error: { code, message } }.

Q: How do you handle polling timeouts and network failures?

A: Per-request timeout (~15s) via AbortController. Overall planning timeout (~90s). Poll interval ~2.5s. Transient poll failures retry until overall timeout; auth failures fail fast. Offline event surfaces a clear UI error.

Q: Why not WebSockets?

A: Hospilot's real UI uses WebSockets, but the assessment says polling is fine. Polling is simpler, easier to test, and good enough for 10-30s planning. WebSockets would be a future improvement for snappier updates.

Q: How would you deploy this?

A: Set Vercel root directory to widget/prerakyadav, add env vars, deploy. Static demo.html + widget assets; /api/session and /api/session/:id as serverless functions. Same-origin requests, no permissive CORS needed.

Q: How do you test without wasting sandbox compute?

A: Unit tests mock global.fetch for login/create/poll/malformed/network cases. Frontend contract tests assert postMessage shape. One optional integration script creates a single real session when explicitly run.

Q: What would you improve with more time?

A: WebSocket readiness updates, stronger observability/request IDs, session expiration and token refresh, and a handshake reply from the iframe confirming session open.

Q: If a reviewer opens your live URL, what should they do?

A: Open the page, use the default/example goal, click Generate Plan, wait for Plan ready, click View Plan, and confirm the iframe shows the created pipeline - not an empty mission screen. That iframe step is the main Part 1 check.

Q: Did you modify the original widget/demo.html?

A: No. The assessment requires copying it into widget/<name>/ and building there. Original widget/demo.html stays untouched; only widget/prerakyadav/demo.html is customized.

7. How To Run & Verify (Cheat Sheet)

Local

```
cd widget/prerakyadav
cp .env.example .env    # set username/password
npm install
npm run dev              # http://localhost:3000
npm test
npm run integration      # optional; creates ONE real session
```

Manual reviewer path

- Open <http://localhost:3000> (or deployed URL)
- Hard refresh if JS changed (Cmd+Shift+R)
- Click example goal -> Generate Plan -> wait -> View Plan
- Success = iframe shows your [CANDIDATE-prerakyadav] plan/pipeline
- Failure pattern = logged in but 'No Pipeline Yet' (handoff/timing issue)

8. Honest Tradeoffs

- Vanilla JS kept Part 1 simple; a framework would help if the UI grew.
- Polling is simpler than WebSockets but slightly less real-time.
- Returning a token to the browser is required by iframe contract; exposure is minimized, not zero.
- Serverless cold starts can add latency on first request after idle.

9. One-Minute Closing Answer

I treated this as a real hospital integration: credentials and Hospilot API calls stay on a same-origin backend, the widget owns UX and iframe handoff, sessions are created with `autonomous=false` and a candidate prefix, readiness is based on pipeline content with timeouts, and the critical demo path is View Plan opening the exact session via `widget_init` `postMessage`. I verified with mocked unit tests plus a controlled live integration, and fixed iframe SPA timing so the plan actually appears.